Day 95 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 5 reference

Watch: The Chinchilla Correction: Unpacking LLM Scaling Laws (Latent AI) | The AI Scaling Problem, Explained (Vivek Gupta)

`day95_scaling_laws.py`, segment by segment, in the order `main()` uses things. Day 90's model and Day 92's schedule, measurement and data code are imported and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: N = weights · D = training characters · L = held-out bits per character · c, a, α = the power law's floor, scale and exponent

Segment 0 — Setup

Imports and settings

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day92_training_mini_gpt as d92
d90 = d92.d90
T, B, H, LAYERS = 64, 32, 4, 2
STEPS, PEAK = 3000, 3e-3
WIDTHS = (16, 32, 64, 128)
FRACTIONS = (1 / 32, 1 / 8, 1 / 2, 1.0)
CHECKS = (250, 500, 1000, 1500, 2000, 2500, 3000)
```

- `d90 = d92.d90` — Day 92 already imported Day 90; reusing that module object avoids importing it twice.
- `FRACTIONS = (1 / 32, 1 / 8, 1 / 2, 1.0)` — fractions of the training text; 1/32 is about 31,000 characters.

Segment 1 — Counting and fitting

count_weights: the model's size by formula

MATH

$$N = V\,d + T\,d + L\,(4d^2 + 2\,d\,f + f + d + 4d) + d\,V + V$$

CODE

```
def count_weights(V, d, layers, t, ff=None):
    ff = ff or 4 * d
    block = 4 * d * d + 2 * d * ff + ff + d + 4 * d
    return V * d + t * d + layers * block + d * V + V
```

- `ff = ff or 4 * d` — 'or' gives the right side when ff is None: the default feed-forward width.
- `4 * d * d + 2 * d * ff + ff + d + 4 * d` — per block: four d × d projections, two feed-forward matrices, their biases, two LayerNorms.

fit_power_law: a grid over the floor, a line for the rest

MATH

$$\log(L - c) = \log a - \alpha \log N; \quad c \text{ chosen to minimise } \Sigma\,(c + a\,N^{-\alpha} - L)^2$$

CODE

```
def fit_power_law(N, L):
    N, L = np.asarray(N, float), np.asarray(L, float)
    best = None
    for c in np.linspace(0, L.min() * 0.999, 400):
        slope, icpt = np.polyfit(np.log(N), np.log(L - c), 1)
        err = np.sum((c + np.exp(icpt) * N ** slope - L) ** 2)
        if best is None or err < best[0]:
            best = (err, c, np.exp(icpt), -slope)
    return best[1:]
```

- `np.linspace(0, L.min() * 0.999, 400)` — 400 candidate floors between 0 and just below the smallest loss.
- `np.polyfit(np.log(N), np.log(L - c), 1)` — a straight line through log(L – c) against log N: slope = –α, intercept = log a.
- `best[1:]` — everything but the error: c, a, alpha.

Segment 2 — The runs

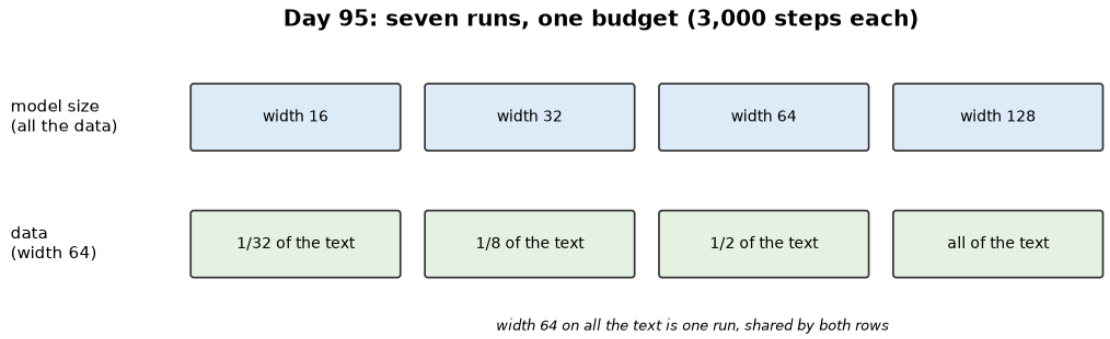
run: one configuration in its own process

CODE

```
def run(job):
    name, d, frac = job
    _, _, vocab, tr, va = d92.data()
    tr = tr[:int(len(tr) * frac)]
    with threadpool_limits(1, "blas"):
        rng = np.random.RandomState(0)
        P = d90.init_gpt(rng, len(vocab), d=d, ff=4 * d, layers=LAYERS, t=T)
        state, curve, t0 = {"t": 0, "m": {}, "v": {}}, [], time.perf_counter()
        for step in range(1, STEPS + 1):
            o = rng.randint(0, len(tr) - T - 1, B)
            x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
            _, G = d90.gpt_loss_grads(x, y, P, h=H)
            d90.adam_step(P, G, state, d92.lr_schedule(step - 1, PEAK, total=STEPS))
            if step in CHECKS:
                curve.append((step, d92.eval_bits(P, va, T, H), d92.eval_bits(P, tr[:len(va)], T,
                    H)))
        n = sum(a.size for _, a in d90.w12.leaves(P))
    return dict(name=name, d=d, frac=frac, chars=len(tr), weights=n, curve=np.array(curve),
        minutes=(time.perf_counter() - t0) / 60)
```

- `tr = tr[:int(len(tr) * frac)]` — the first part of the training text: less data, same everything else.
- `d92.lr_schedule(step - 1, PEAK, total=STEPS)` — Day 92's warm-up and cosine decay.
- `d92.eval_bits(P, tr[:len(va)], T, H)` — the same measurement on training text: the overfitting check.

SYSTEM DIAGRAM



Seven runs, one budget.

Segment 3 — Fitting and reporting

demo_fits: final losses and best checkpoints

CODE

```
for name, rs, key, pick in (("model size", sizes, "weights", "final"), ("data", datas, "chars",
    "final"),
                            ("data, best checkpoint", datas, "chars", "best")):
    x = np.array([r[key] for r in rs])
    L = np.array([r["curve"][-1, 1] if pick == "final" else r["curve"][:, 1].min() for r in rs])
    c, a, al = fit_power_law(x, L)
    loo = [fit_power_law(np.delete(x, i), np.delete(L, i))[2] for i in range(len(x))]
    out[name] = (x, L, (c, a, al))
    print(f"\n{name}: x = {', '.join(str(int(v)) for v in x)}; held-out bits = {',
        '.join(f'{v:.3f}' for v in L)}")
    print(f"  fit: c = {c:.3f}, alpha = {al:.3f}; leaving one point out, alpha = " + ",
        ".join(f"{v:.2f}" for v in loo))
    print(f"  largest gap between the fit and a point: {np.abs(c + a * x ** -al - L).max():.3f}
        bits")
```

- `r["curve"][-1, 1] if pick == "final" else r["curve"][:, 1].min()` — the last held-out loss, or the lowest one at any check.
- `np.delete(x, i)` — a copy without entry i: the leave-one-out fits.
- `np.abs(c + a * x ** -al - L).max()` — the worst miss of the fitted curve.

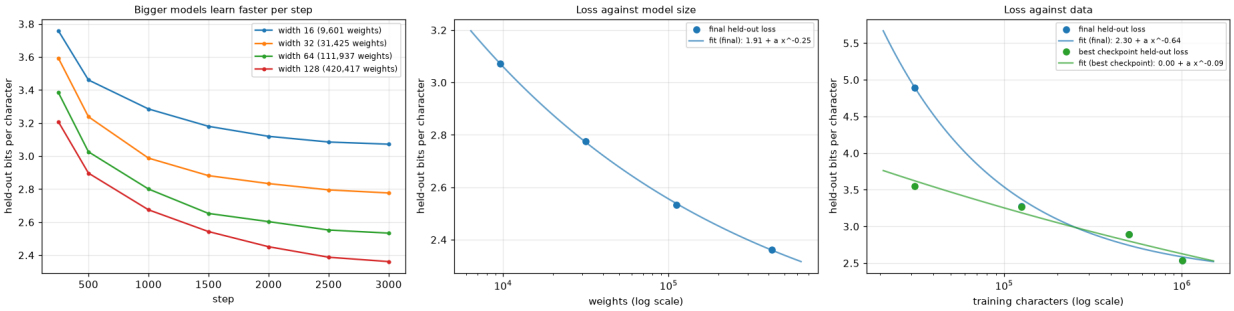
demo_overfit: passes over the data

CODE

```
passes = STEPS * B * T / r["chars"]
print(f"{r['frac']:<16g}{r['chars']:>11}{passes:>15.1f}{r['curve'][-1, 2]:>12.3f}{r['curve'][-1,
    1]:>10.3f}"
      f"{r['curve'][-1, 1] - r['curve'][-1, 2]:>7.3f}")
```

- `STEPS * B * T / r["chars"]` — characters read in training divided by characters available: how many times each was seen.
- `r['curve'][-1, 1] - r['curve'][-1, 2]` — held-out minus training text: the gap.

REAL GRAPH



Real graph from the script: curves by size, and loss against size and against data.

Quick Syntax Glossary — new or reinforced constructs

- `np.polyfit(x, y, 1)` — least-squares straight line: slope and intercept.
- `np.linspace(a, b, n)` — n evenly spaced values from a to b.
- `np.logspace(a, b, n)` — n values evenly spaced on a log scale, 10^a to 10^b.
- `np.delete(a, i)` — a copy of a without entry i.
- `x or default` — x if it is truthy, otherwise the default.
- `ax.set_xscale("log")` — a logarithmic axis, on which power laws look like gentle curves or lines.

End of Day 95 syntax reference. For the experiments and what they showed, see the Day 95 complete guide.